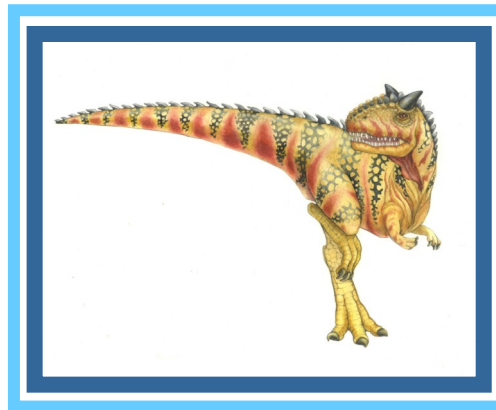


Chapter 6: Deadlocks





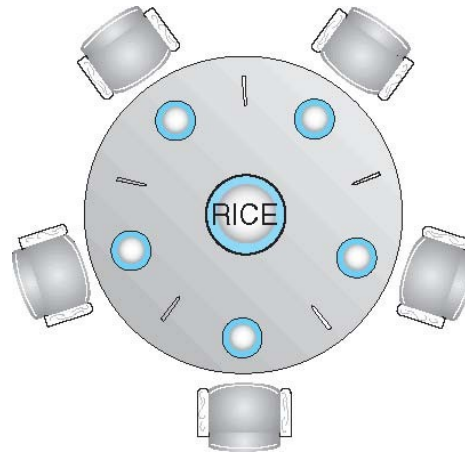
Chapter 6: Deadlocks

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock





Dining-Philosophers Problem



- Philosophers spend their lives alternating thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers
 - Shared data
 - ▶ Bowl of rice (data set)
 - ▶ Semaphore **chopstick** [5] initialized to 1





Dining-Philosophers Problem Algorithm

- The structure of Philosopher *i*:

```
do {  
    wait (chopstick[i] );  
    wait (chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- What is the problem with this algorithm?





Problems with Semaphores

- ❑ Incorrect use of semaphore operations:
 - ❑ signal (mutex) wait (mutex)
 - ❑ wait (mutex) ... wait (mutex)
 - ❑ Omitting of wait (mutex) or signal (mutex) (or both)
- ❑ Deadlock and starvation are possible.





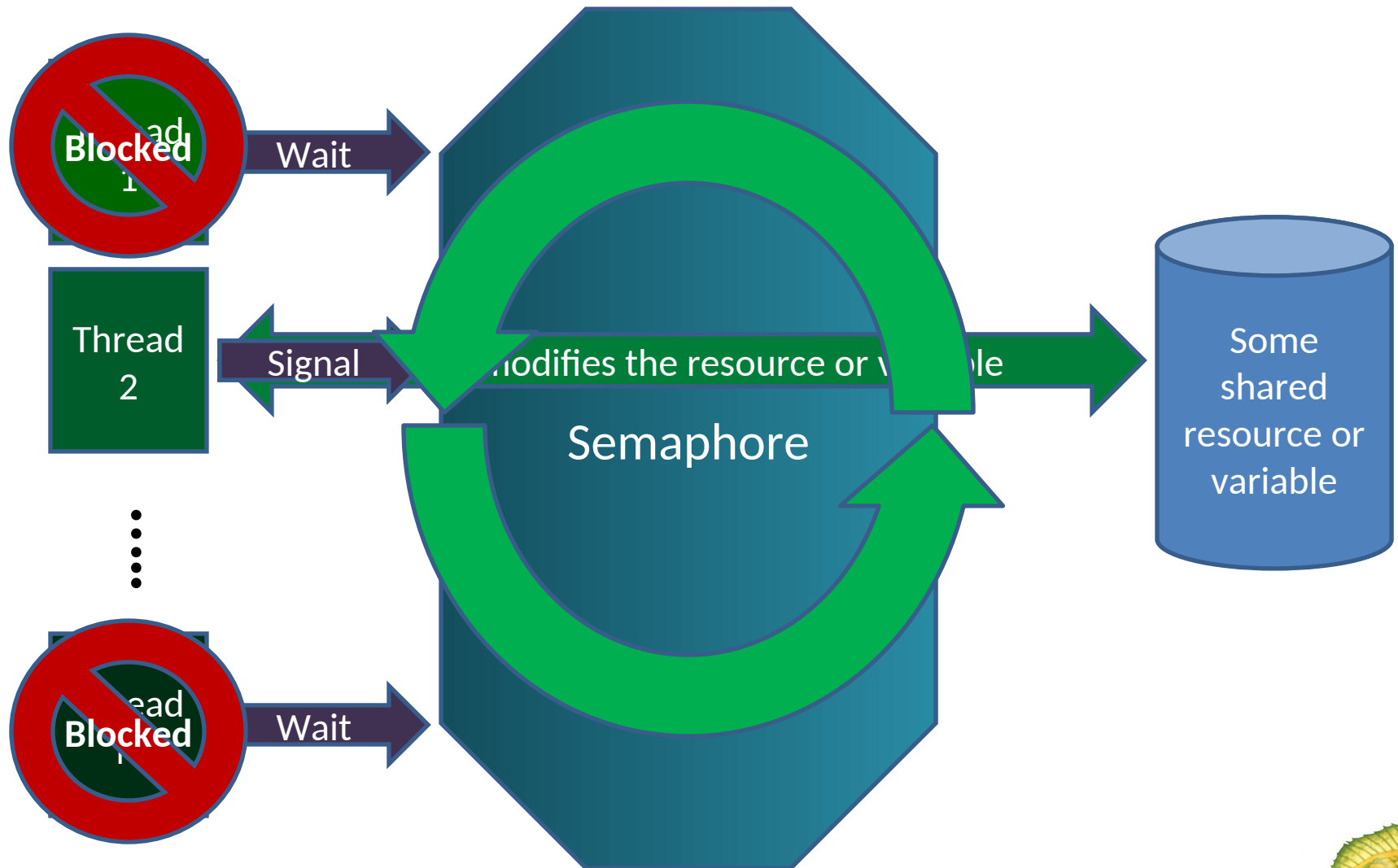
Starvation

- Semaphores do not wake blocked processes in any specific order
 - In other words, it does not use a FIFO queue
 - This means, ***starvation*** of a process can occur
- When starvation occurs, it means a process doesn't get the chance to run





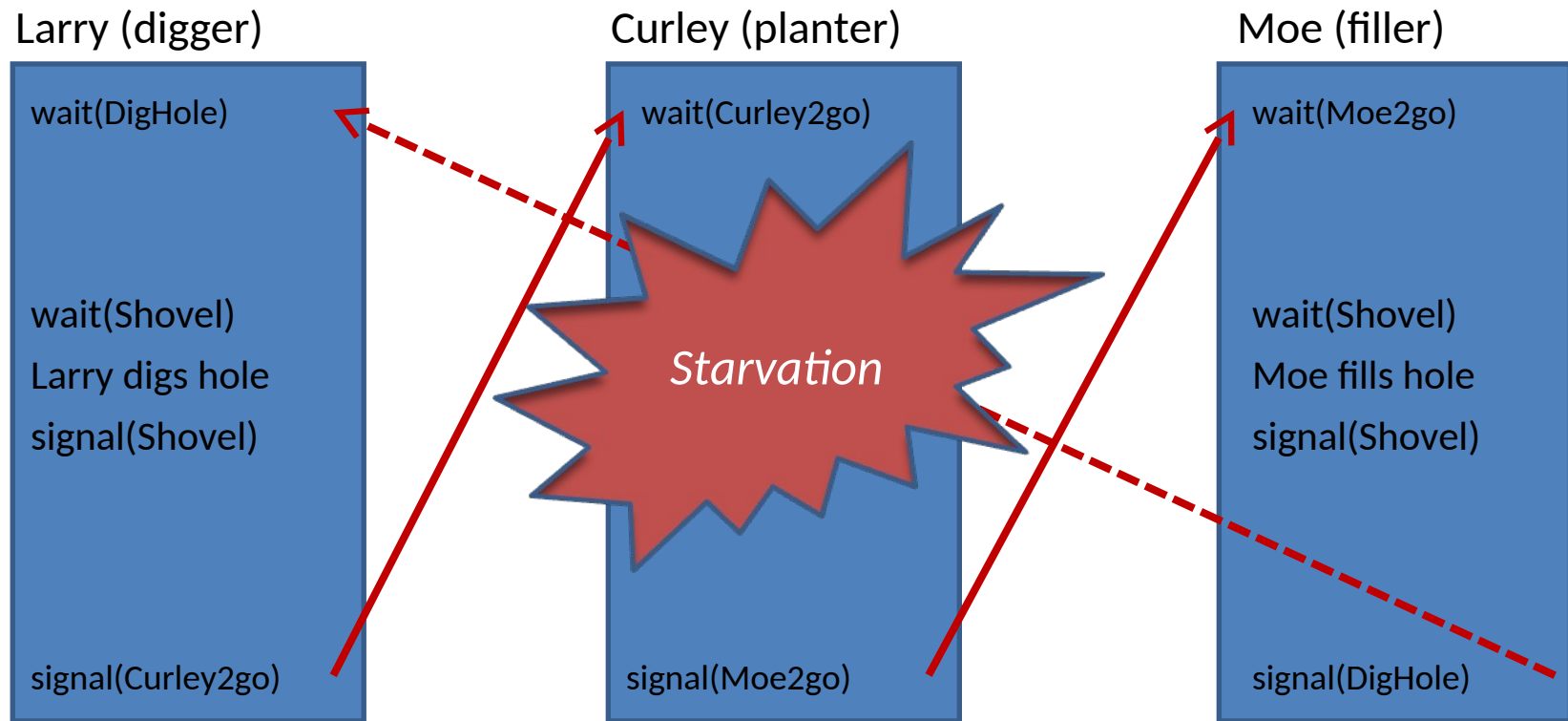
Visualize Starvation





Starvation Example

Have 4 semaphores: **DigHole = 0**, Curley2go = 0, Moe2go = 0, and Shovel = 1.





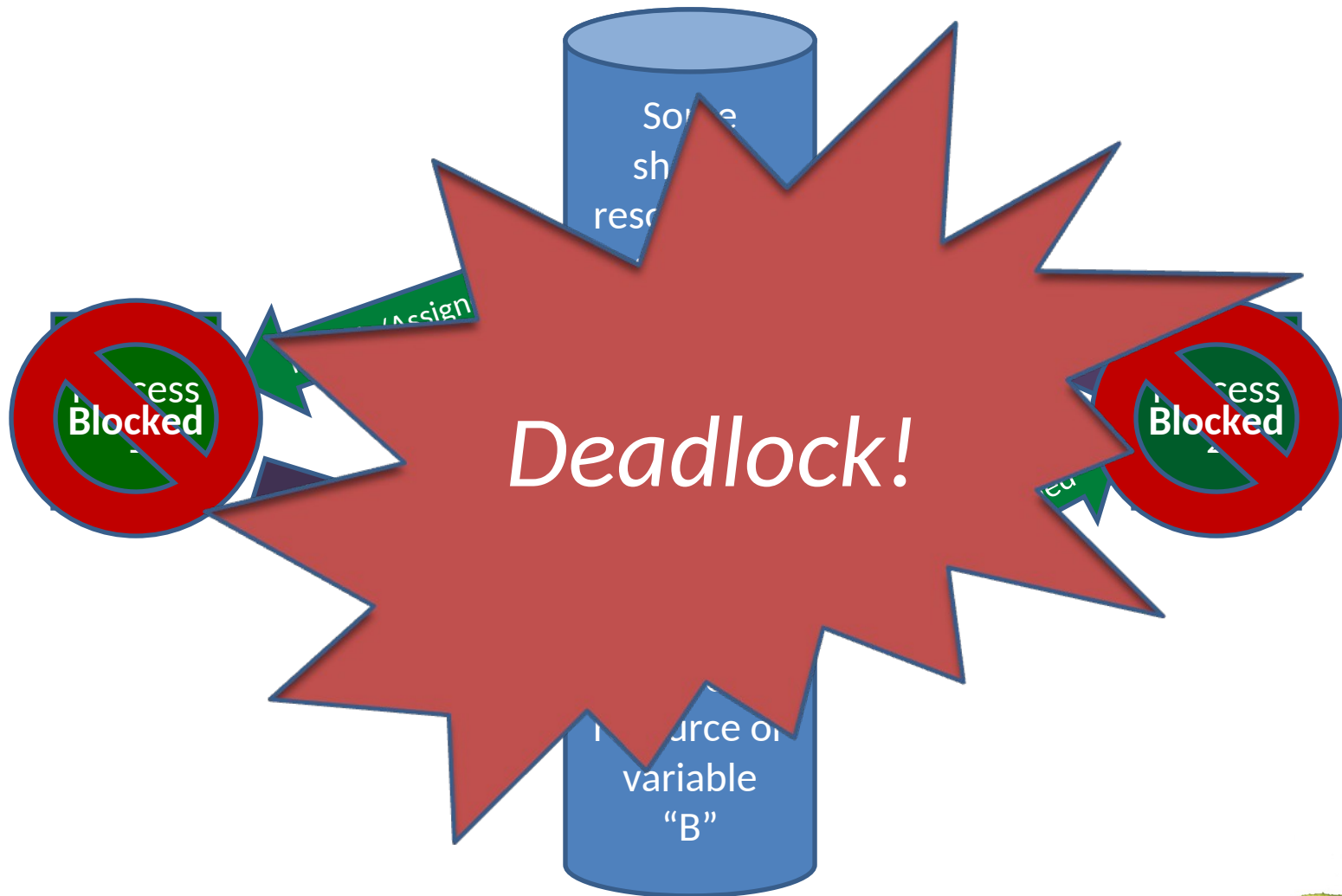
Deadlocks

- Semaphores do not prevent deadlock
 - They *can* prevent **deadlock**, if used cleverly (this will be discussed more in Lecture – “Dining Philosophers”)
- When deadlocks occur, it means multiple processes are executing forever.





Visualize a Deadlock





Deadlock Example

P_0

```
wait(S);  
wait(Q);  
.  
.  
.  
signal(S);  
signal(Q);
```

P_1

```
wait(Q);  
wait(S);  
.  
.  
.  
signal(Q);  
signal(S);
```





Deadlock in Dining-Philosopher

- Deadlock:
 - Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.
- Deadlock handling solutions:
 - Allow at most 4 philosophers to be sitting simultaneously at the table.
 - Allow a philosopher to pick up the forks only if both are available (picking must be done in a critical section).
 - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .





System Model

- System consists of resources
- Resource types $R_1, R_2, \dots, R_m$
 - CPU cycles, memory space, I/O devices*
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**





Resource-Allocation Graph

A set of vertices V and a set of edges E .

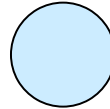
- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - ▶ Represented as **circle**
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
 - ▶ Represented as **rectangle** and **dots** (instances)
- **request edge** – directed edge $P_i \rightarrow R_j$
 - It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.
- **assignment edge** – directed edge $R_j \rightarrow P_i$
 - it signifies that an instance of resource type R_j has been allocated to process P_i .





Resource-Allocation Graph (Cont.)

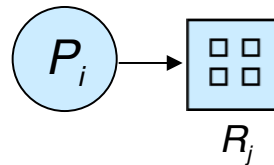
□ Process



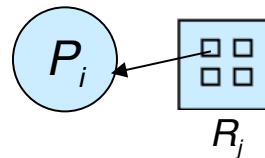
□ Resource Type with 4 instances



□ P_i requests instance of R_j



□ P_i is holding an instance of R_j





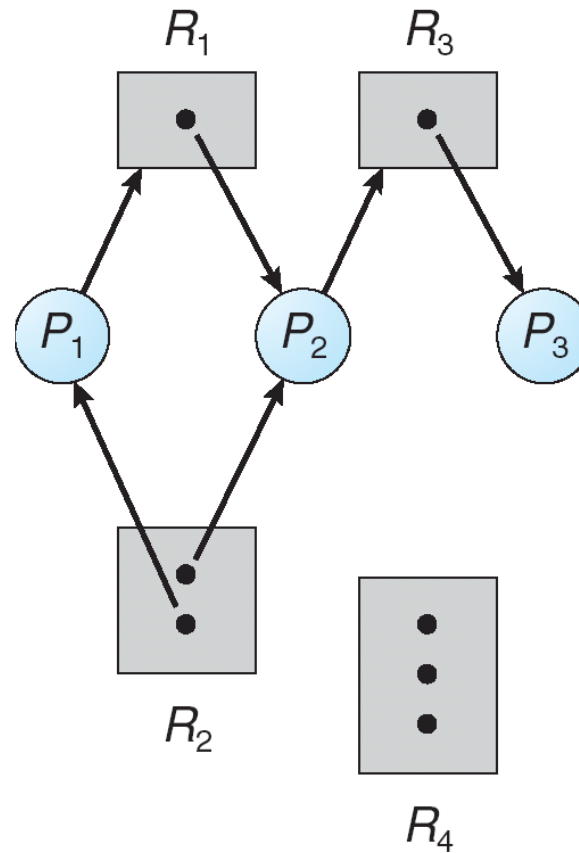
Resource-Allocation Graph (cont)

- When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource-allocation graph.
- When this request can be fulfilled, the request edge is ***instantaneously*** transformed to an assignment edge.
- When the process no longer needs access to the resource, it releases the resource. As a result, the assignment edge is deleted.





Example of a Resource Allocation Graph





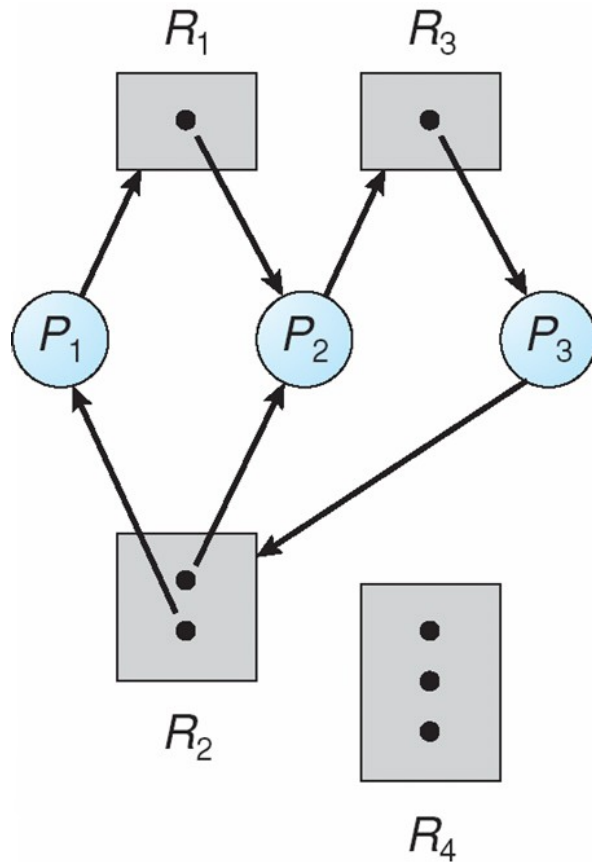
Deadlocks

- In the resource-allocation graph:
 - If the graph contains no cycles, then no process in the system is deadlocked.
 - If the graph does contain a cycle, then a deadlock may exist.
 - ▶ If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. Each process involved in the cycle is deadlocked
 - ▶ If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred

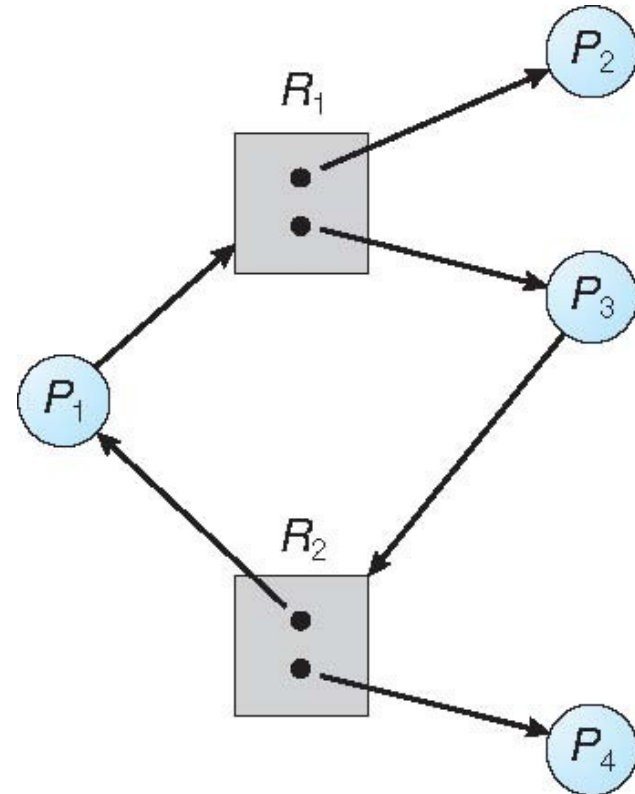




Deadlocks Examples



Graph With A Cycle That is A
Deadlock



Graph With A Cycle But No Deadlock





Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock





Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention (user/process responsibility, limit requests, may have side effect)
 - **Deadlock avoidance**
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX





Deadlock Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph

- Multiple instances of a resource type
 - Use the banker's algorithm





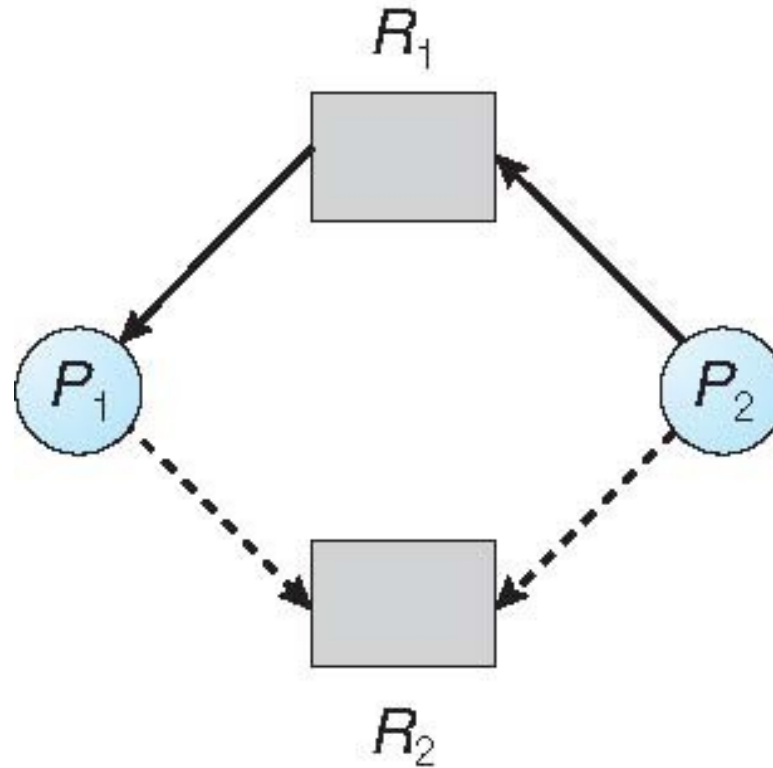
Resource-Allocation Graph Scheme

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system





Resource-Allocation Graph





Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph
- We check for safety by using a cycle-detection algorithm. An algorithm for detecting a cycle in this graph requires an order of n^2 operations, where n is the number of processes in the system.
 - If no cycle exists, then the allocation of the resource will leave the system in a safe state.
 - If a cycle is found, then the allocation will put the system in an unsafe state. In that case, process P_i will have to wait for its requests to be satisfied.





Unsafe State In Resource-Allocation Graph

